The Grand Stay

Luxury Room Booking

Version 4.0 • Final Project Report

Submitted By: M Talal Ashraf

Registration No: 232201052

Class / Section: BSCS-6B

Submitted To: Sir Uzair Hassan

Domain: Room Booking

Date: April 15, 2026

1 Executive Summary

Project at a Glance

The Grand Stay is a fully automated Hotel Management System built on **C# .NET Core** and **MongoDB**. This system eliminates manual paper-based booking by providing a complete digital pipeline — from guest check-in to automated billing and checkout.

The system has two primary design goals:

1. **Data Integrity** — Every booking transaction is atomic. Guest data and room status are saved simultaneously, making double booking impossible.
2. **Dynamic Resource Allocation** — Rooms are filtered from the database in real-time, displaying only those that are available.

2 System Architecture

The project is divided into **4 layers** (N-Tier Design).

2.1 N-Tier Architecture

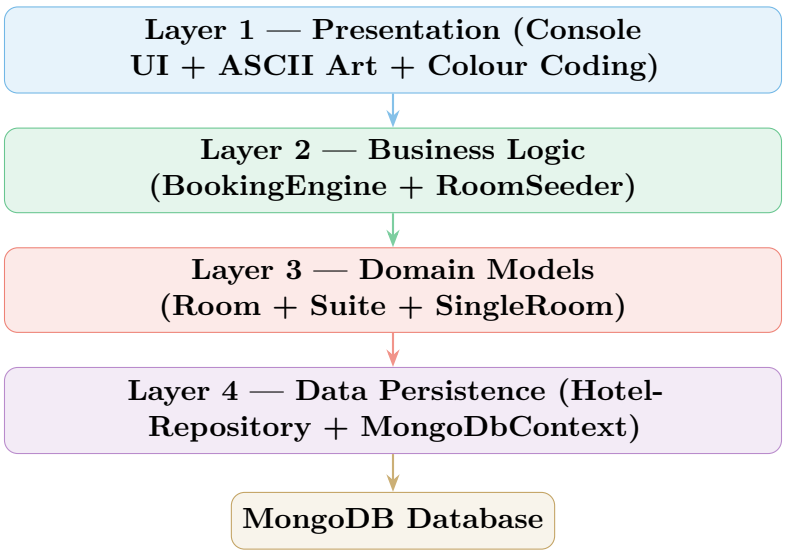

Figure 1: N-Tier Architecture of The Grand Stay HMS

2.2 Layer Descriptions

Console UI utilizing ASCII art headers, a `CenterText()` function for alignment, and contextual colour coding. **Green** = success, **Red** = warning, **Yellow** = invoice.

BookingEngine — The core brain that filters rooms and calculates pricing logic.

RoomSeeder — Inserts 50 unique rooms into MongoDB during the first launch.

Room (abstract) defines common fields. **Suite** and **SingleRoom** inherit from this and add their own specific properties.

MongoDbContext handles the database connection. **HotelRepository** implements the Repository Pattern — exposing only clean Add/Update/Fetch methods.

3 Class & Object Model (UML)

3.1 Inheritance Hierarchy

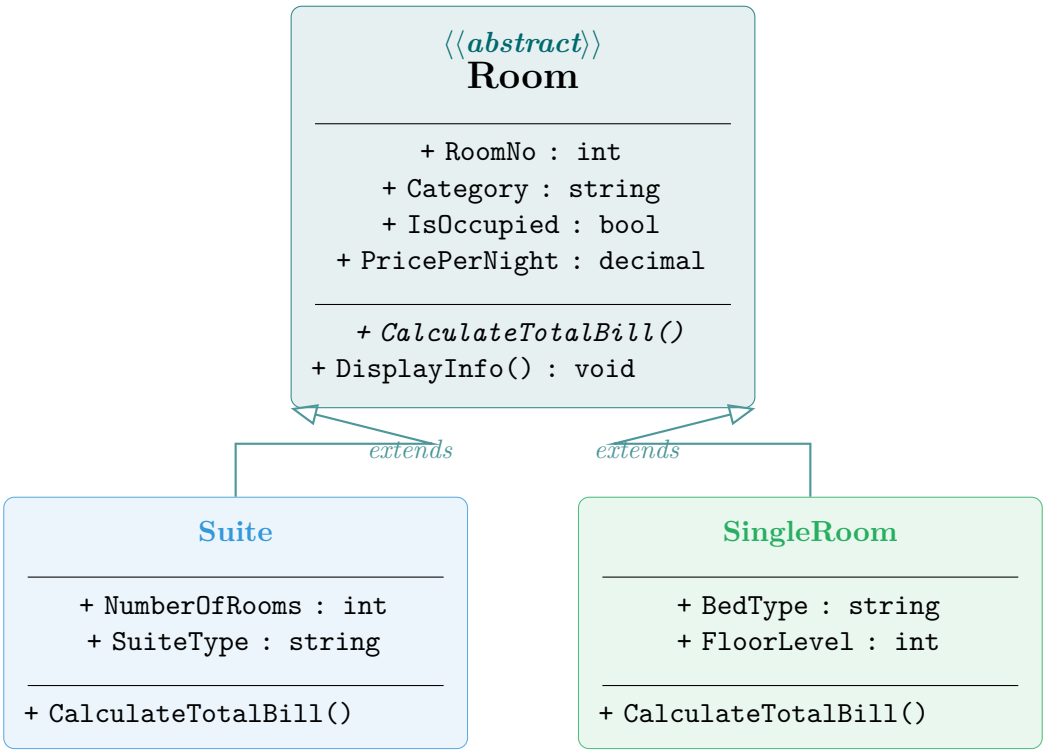

Figure 2: UML Class Diagram — Room Inheritance Hierarchy

3.2 Component Relationship

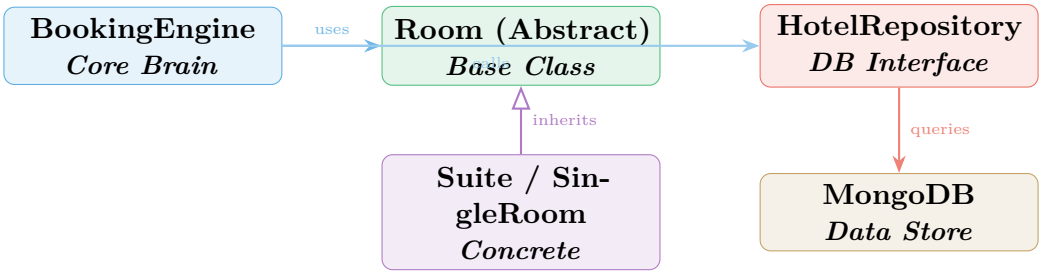

Figure 3: Component Relationship Diagram

4 System Working Flow

4.1 Complete Flowchart

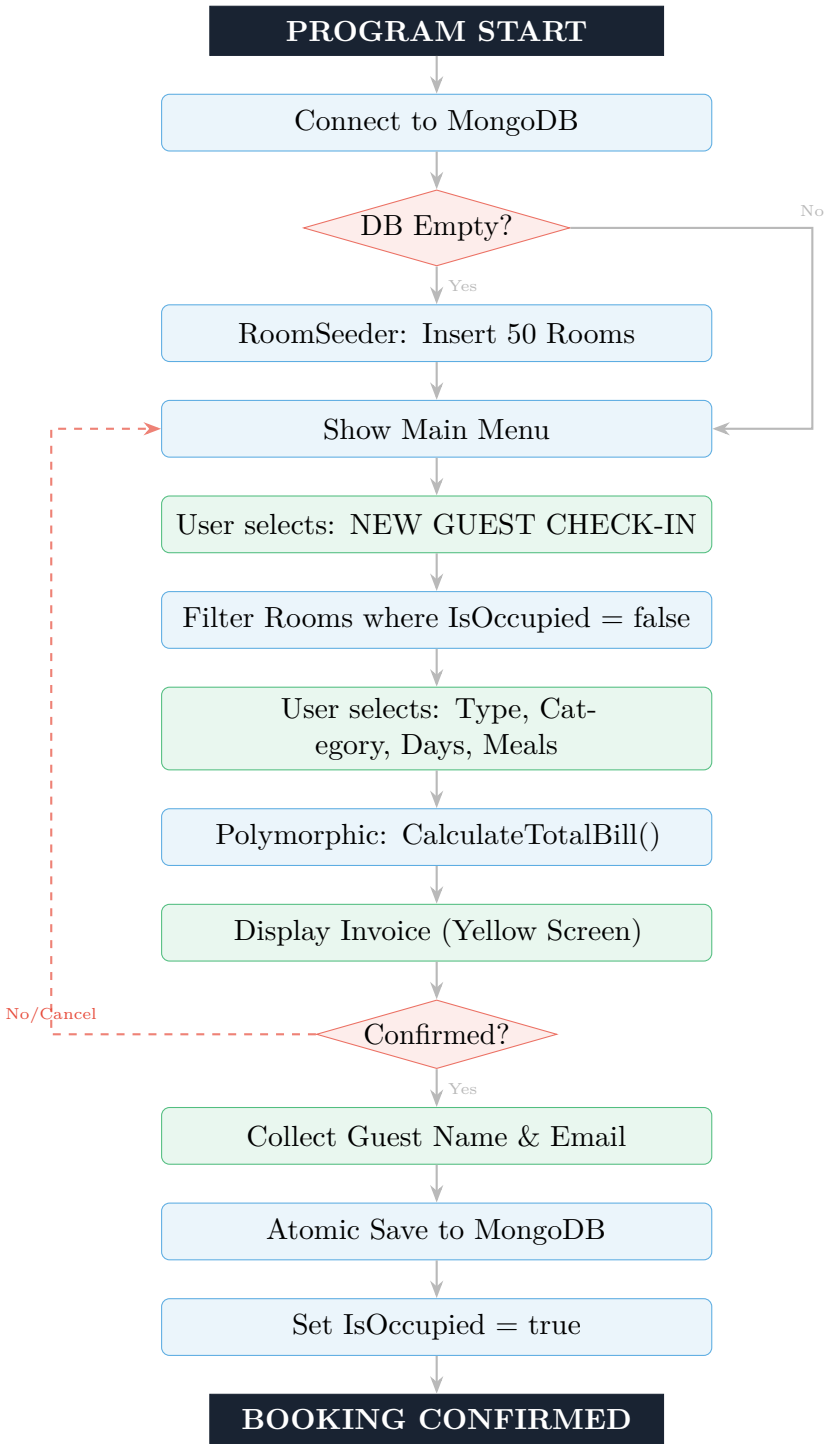

Figure 4: End-to-End System Flow

4.2 Phase-by-Phase Description

1. The program starts and establishes a connection with MongoDB.
2. **RoomSeeder** checks the status: if the database is empty, it inserts 50 rooms; otherwise, it skips seeding.

1. The user selects "New Guest Check-In" from the main menu.
2. The system fetches **only available rooms** (`IsOccupied = false`).
3. The user chooses the room type (Suite/Room), category, stay duration, and meal plan.

1. **Polymorphism** calls the correct version of `CalculateTotalBill()` at runtime.
2. Invoice screen displays:
   $\text{Total} = (\text{Base} + \text{Surcharge}) \times \text{Days} + \text{Meal Cost}$
3. The user has the option to confirm or cancel the booking.

1. Guest name and email are collected *only after confirmation* to prevent orphan records.
2. **Atomic transaction**: Guest data is saved and room status is updated simultaneously.
3. Booked rooms are marked as `IsOccupied = true`.

5 OOP Concepts Implementation

5.1 Four Pillars Diagram

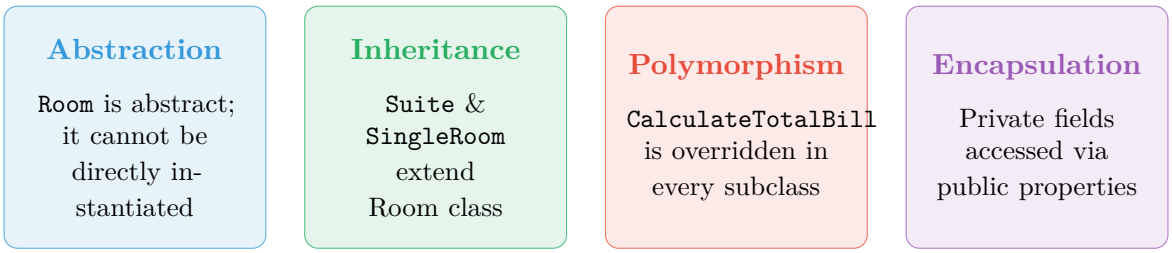

Figure 5: Four Pillars of OOP Applied in The Grand Stay

5.2 Implementation Table

OOP Concept	Application in Project
Abstraction	The `Room` class is abstract. It serves only as a blueprint and cannot be created using `new Room()`.
Inheritance	Both `Suite` and `SingleRoom` inherit from `Room`. Common attributes are centralized in the base class.
Polymorphism	The `CalculateTotalBill()` method is declared in `Room` and overridden in both subclasses. C# calls the correct version at runtime.
Encapsulation	All fields use `private set`. Access is restricted through public properties to prevent unauthorized data manipulation.

Table 1: OOP Concepts — Project Applications

6 Database Design

6.1 MongoDB Document Schema

Collection: `rooms`

```
{
  "_id"  : ObjectId,
  "RoomNo"  : int,
  "Category" : "Luxury" | "Basic",
  "Type"  : "Suite" | "SingleRoom",
  "BedType"  : string // SingleRoom only,
  "NumberOfRooms" : int // Suite only,
  "PricePerNight" : decimal,
  "IsOccupied"  : bool,
  "GuestName"  : string | null,
  "GuestEmail"  : string | null,
  "MealPlan"  : [ "Breakfast", "Dinner" ],
  "StayDays"  : int
}
```

Figure 6: MongoDB Document Structure for a Room Record

6.2 Repository Pattern Flow

Figure 7: Repository Pattern — Business Logic Isolated from Database

7 User Interface Design

7.1 UI Principles

Principle	Description
ASCII Aesthetics	Large ASCII art headers provide a premium hotel lobby feel in the console.
Visual Hierarchy	A `CenterText()` function ensures all content is balanced across the screen.
Colour Coding	**Green** = Confirmed **Red** = Error **Yellow** = Invoice
Contextual Feedback	Immediate visual confirmation is provided after every user action.

Table 2: UI/UX Design Decisions

7.2 Screen Navigation Flow

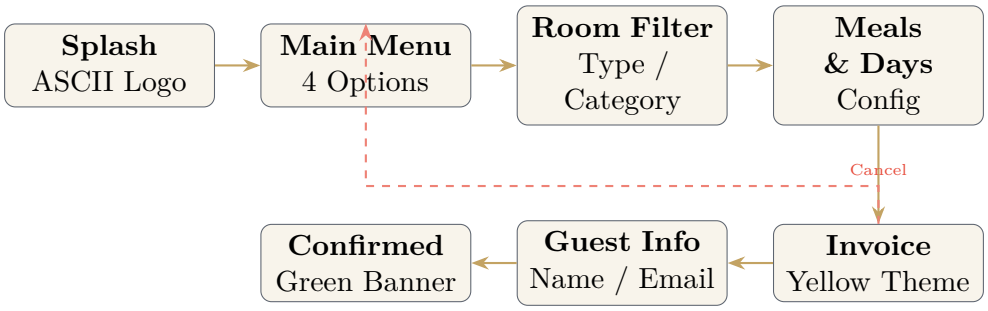

Figure 8: Screen-by-Screen Navigation Flow

8 Conclusion

Key Takeaways

- **Real-World Problem Solving** — Addressed double bookings, manual invoicing, and data inconsistency.
- **Modern Stack** — C# .NET Core combined with MongoDB provides a flexible and strongly-typed environment.
- **Clean Architecture** — Any layer can be independently swapped without affecting the rest of the codebase.
- **Future Ready** — Already structured for easy conversion into an ASP.NET Core Web API or a mobile application.